

Intra AUST Programming Contest – Spring 2025

Preliminary Round

Official Editorial

June 15, 2026

Contents

1	Problem A: RedX Again	2
1.1	Problem Summary	2
1.2	Solution Explanation	2
1.2.1	Approach 1: Cross-Multiplication Slopes	2
1.2.2	Approach 2: GCD Slope Frequency Hashing	2
2	Problem B: Lichu Chor	2
2.1	Problem Summary	2
2.2	Solution Explanation	3
3	Problem C: Zoro’s Grand Tour of AUST	3
3.1	Problem Summary	3
3.2	Solution Explanation	3
4	Problem D: Game of All	3
4.1	Problem Summary	3
4.2	Solution Explanation	4
5	Problem E: Street Lights and Mex	4
5.1	Problem Summary	4
5.2	Solution Explanation	4
6	Problem F: Rakib’s Social Distance Algorithm	5
6.1	Problem Summary	5
6.2	Solution Explanation	5
7	Problem G: The Corrupted RAID Array	5
7.1	Problem Summary	5
7.2	Solution Explanation	5
8	Problem H: The Salamanca Sunset	6
8.1	Problem Summary	6
8.2	Solution Explanations	6
8.2.1	Approach 1: Closed-Form Math Solution	6
8.2.2	Approach 2: Binary Search Solution	6

1 Problem A: RedX Again

- **Author:** [Nayem Uddin](#)

1.1 Problem Summary

Given the coordinates of N students scattered across a 2D grid and the exact coordinate of the Night King (X, Y) , determine the maximum number of students that can be eliminated with a single straight line spear throw starting from (X, Y) and continuing infinitely along that ray.

1.2 Solution Explanation

The Core Idea: Imagine you are standing at the Night King's position, looking out into a field where students are standing. If two students are standing exactly behind one another from your point of view, a single straight spear throw can pierce through both of them. We want to find a direction to look that lines up as many students as possible.

1.2.1 Approach 1: Cross-Multiplication Slopes

You pick one student and draw a line from the Night King through that student. Then, you look at every other student one by one to see if they lie perfectly on that same line and in that same direction. You repeat this for every single student to see which target gives you the highest count.

- **Time Complexity:** $\mathcal{O}(T \cdot N^2)$
- **Space Complexity:** $\mathcal{O}(N)$
- **Source Code:** [Link](#)

1.2.2 Approach 2: GCD Slope Frequency Hashing

Instead of relying on a nested loop, you find the direction vector (the slope) from the Night King to each student. By dividing the horizontal change (dx) and vertical change (dy) by their Greatest Common Divisor (gcd), you reduce every angle into its simplest fraction form. Students with identical simplified fractions are standing on the exact same line of sight. You group them using a frequency count (`map`) to find the most populated line instantly.

- **Time Complexity:** $\mathcal{O}(T \cdot N \log N)$
- **Space Complexity:** $\mathcal{O}(N)$
- **Source Code:** [Link](#)

2 Problem B: Lichu Chor

- **Author:** [Hridoy Nandi](#)

2.1 Problem Summary

A bag contains N distinct items. In a single move, if r items remain, a subset is picked uniformly at random out of 2^r possibilities. If the chosen configuration is one of M pre-designated illegal subsets, the move has no effect. Otherwise, all elements in the selected subset are cleared from the bag. Compute the expected number of moves required to empty the bag modulo 998244353.

2.2 Solution Explanation

The Core Idea: You have a set of items, and you want to know how many turns on average it will take to remove all of them. Each turn, you randomly pick a subset of the remaining items. If it's a valid choice, you take them out (and your set of items shrinks). If it's a forbidden/illegal choice, nothing happens, and you waste a turn.

Because N is very small ($N \leq 18$), we can represent our current remaining items as a binary number (a bitmask), where a 1 means the item is still in the bag and 0 means it's gone. We calculate the answer from the smallest groups up to the full set. For a specific group of remaining items (**mask**), we look at all its possible subsets (**submask**).

If a subset is valid, it successfully shrinks our item set down to a smaller state (**mask** $\wedge$ **submask**). We look up the expected turns already calculated for that smaller state, add 1 for the current step, and average them out based on how many valid choices we have. If no valid subsets can be picked to make progress, we are trapped forever, so the code outputs -1.

- **Time Complexity:** $\mathcal{O}(T \cdot (3^N + M))$
- **Space Complexity:** $\mathcal{O}(2^N)$
- **Source Code:** [Link](#)

3 Problem C: Zoro's Grand Tour of AUST

- **Author:** [Rakib Hasan](#)

3.1 Problem Summary

Follow a precise series of spatial grid operations on a 2-row, 26-element landmark layout to decipher a secret 7-letter destination string.

3.2 Solution Explanation

The Core Idea: This is a straightforward tracing game on a fixed character board. You start your character cursor at the very top-left letter (**A**) and blindly move around according to fixed directions (move right, drop down a row, move left, etc.).

Since there is no input data and the instructions never change, the path is completely fixed. Following the steps lands you on the letters **U**, **S**, **T**, **P**, **I**, and **C** in that exact sequence. Because the final answer is always the same, the program is optimized to directly print the secret decoded string: "AUSTPIC".

- **Time Complexity:** $\mathcal{O}(1)$
- **Space Complexity:** $\mathcal{O}(1)$
- **Source Code:** [Link](#)

4 Problem D: Game of All

- **Author:** [Sajid Abdullah](#)

4.1 Problem Summary

An array of N integers is given. We select any subsequence while preserving the original relative array index order. A fixed, cyclic pattern of arithmetic/bitwise operations (+, &, |, ^) is sequentially calculated over the chosen elements. Find the maximum final value achievable.

4.2 Solution Explanation

The Core Idea: You are given a list of numbers. You must pick some numbers from left to right. As you pick them, you must combine them using a strict, repeating cycle of math operations: the 1st operation must be $+$, the 2nd must be $\&$ (AND), the 3rd must be $|$ (OR), and the 4th must be $\wedge$ (XOR). Then the loop restarts back at $+$. We want to find the collection of numbers that calculates out to the largest possible final result.

Since you are given at most 20 numbers, the total number of ways to choose a subsequence is 2^{20} (about 1 million combinations). The code uses a loop from 1 to $2^N - 1$ to simulate every possible configuration using binary digits. If the j -th bit of our loop counter is 1, we include the j -th number. For each combination, it processes the selected numbers from left to right, strictly cycling the math operations using a counter. It keeps track of the running score and saves the highest value it manages to find.

- **Time Complexity:** $\mathcal{O}(T \cdot 2^N \cdot N)$
- **Space Complexity:** $\mathcal{O}(N)$
- **Source Code:** [Link](#)

5 Problem E: Street Lights and Mex

- **Author:** Zubaer Mahmud

5.1 Problem Summary

Given an array containing N distinct elements, compute the cumulative sum of the Minimum Excluded Value (MEX) evaluated across all possible contiguous subarrays.

5.2 Solution Explanation

The Core Idea: The MEX of an array is the smallest non-negative integer $(0, 1, 2, \dots)$ that is missing. We want to find the sum of MEX values for all possible contiguous windows (subarrays) of our array. Because all elements are unique, a window will have a MEX value of *at least* i if and only if it manages to swallow up all the positions of the numbers $0, 1, 2, \dots, i - 1$.

The code records the positions of all numbers inside a lookup array `pos`. It iterates through numbers $i = 0, 1, 2, \dots$ and tracks an expanding boundary box (from index `mn` to index `mx`) that is required to contain all numbers from 0 up to i .

- For a fixed value i , any window that completely covers the space between `mn` and `mx` will successfully achieve a MEX value higher than i .
- The number of choices for the left edge of this window is `mn`.
- The number of choices for the right edge is $N - \text{mx} + 1$.
- Multiplying these two values tells us exactly how many subarrays contain all elements up to i . We sum these counts up linearly. If a number i is missing entirely from the input, the chain breaks, and we stop.
- **Time Complexity:** $\mathcal{O}(T \cdot N)$
- **Space Complexity:** $\mathcal{O}(N)$
- **Source Code:** [Link](#)

6 Problem F: Rakib's Social Distance Algorithm

- **Author:** Rakib Hasan

6.1 Problem Summary

Evaluate an $N \times 4$ grid layout representing carriage seating. Rows containing a locked seat marker (*) are completely unusable. Strangers have a willingness score; Rakib can swap with them only if their score is $\geq K$. Identify the optimal row to secure entirely based on: minimum strangers approached, maximum score sum, and smallest 1-based row index.

6.2 Solution Explanation

The Core Idea: You have a grid of train seats with 4 seats per row. Some seats have friends (#), some are broken or blocked (*), and others have strangers with a friendliness score. Rakib wants to find a row where he can sit with his friends. He can only convince a stranger to swap seats if their friendliness score is at least K . We need to filter out bad rows and rank the good ones.

The code reads the train grid row by row. For each row, it loops through the 4 seats. If it sees a *, that entire row is instantly thrown away. If it sees a stranger whose score is smaller than K , that row is also thrown away. If the row passes these checks, it counts how many valid strangers are sitting there (which equals the number of people Rakib must talk to) and sums up their scores.

Finally, it sorts all the passing rows using a simple priority checklist: fewer strangers to approach is best; if there is a tie, pick the row with the higher total friendliness score; if there is still a tie, pick the row that shows up earliest on the train (smallest row index). If no row works, it prints "Rag korla?".

- **Time Complexity:** $\mathcal{O}(T \cdot N \log N)$
- **Space Complexity:** $\mathcal{O}(N)$
- **Source Code:** [Link](#)

7 Problem G: The Corrupted RAID Array

- **Author:** Zahinul Haque

7.1 Problem Summary

An array of N values is processed using variable partition block sizes $L \in [1, N]$. For a choice L , the array is sliced sequentially into chunks of length L , discarding trailing items. Compute the collection of distinct block sums generated across all valid configurations of L , and answer Q queries identifying how many unique sums are $\leq X$.

7.2 Solution Explanation

The Core Idea: You slice an array into equal-sized chunks of length L . Any extra elements at the very end that don't make a full chunk of size L are thrown away. You calculate the sum of each chunk. You do this for every single possible chunk size L from 1 all the way to N , and collect all the resulting sums into a big bucket. Finally, you answer queries asking how many *unique* numbers in that bucket are less than or equal to a value X .

Slicing an array means jumping forward by steps of L . Thanks to the math of harmonic series ($\frac{N}{1} + \frac{N}{2} + \frac{N}{3} + \dots$), the total number of chunks we will ever look at is tightly bounded to

$\mathcal{O}(N \log N)$. To calculate the sum of a chunk instantly without looping through its elements, the code uses a **Prefix Sum Array** (`pref`), where the sum of any chunk can be found by a simple subtraction: `pref[right] - pref[left]`. It collects all chunk sums, sorts them, and uses `unique` to discard duplicates. For each query value X , it uses binary search (`upper_bound`) to quickly count how many distinct numbers in our sorted bucket fall under the threshold X .

- **Time Complexity:** $\mathcal{O}(T \cdot (N \log N + Q \log N))$
- **Space Complexity:** $\mathcal{O}(N \log N)$
- **Source Code:** [Link](#)

8 Problem H: The Salamanca Sunset

- **Author:** [Nayem Uddin](#)

8.1 Problem Summary

Given two positive initialization values A and B (with $B < A$), both values grow by exactly 1 unit per year. Calculate the exact count of years $Y \geq 0$ required to satisfy the equation $A + Y = 2(B + Y)$. If no valid non-negative integer solution can be reached, output -1 .

8.2 Solution Explanations

The Core Idea: Gustavo has A dollars and Hector has B dollars ($B < A$). Every year, both of their revenues grow by exactly 1 dollar. Gustavo wants to know how many years it will take for his revenue to become exactly double Hector's revenue ($A + Y = 2 \cdot (B + Y)$).

8.2.1 Approach 1: Closed-Form Math Solution

If you solve the equation using basic algebra, it simplifies directly:

$$A + Y = 2B + 2Y \implies Y = A - 2B$$

So the answer is just $A - 2B$. Since years cannot run backward, if $A - 2B$ turns out to be a negative number, it means the opportunity has already passed and it will never happen again, so we output -1 .

- **Time Complexity:** $\mathcal{O}(1)$
- **Space Complexity:** $\mathcal{O}(1)$
- **Source Code:** [Link](#)

8.2.2 Approach 2: Binary Search Solution

Alternatively, because Gustavo's revenue grows slower relative to *double* Hector's revenue over time, the gap closes continuously. The code guesses a year in the middle of a massive search range (`mid`). If the guess makes Gustavo's revenue too small compared to double Hector's, it knows it needs to look at later years. If it's too big, it looks at earlier years, narrowing down the exact year matching the condition.

- **Time Complexity:** $\mathcal{O}(T \cdot \log(\text{MAX_Y}))$
- **Space Complexity:** $\mathcal{O}(1)$
- **Source Code:** [Link](#)

Note

Please note that the codes and explanations provided in this editorial are not the only solutions to these problems. Each problem can have multiple solutions, alternative logic patterns, and more optimized approaches. Participants are highly encouraged to explore different methods and discover even more efficient implementations.